

CSE234 Week 11: Big 3 and Generic Types

C++ provides polymorphism in three main ways: function overloading, generic types, and inheritance. Function overloading should have been covered in the prior course. This week, we will focus on generic types. Inheritance will be covered in a couple of weeks. In C++, you are expected to manage the memory of any dynamically allocated fields you create. The Big 3 is a term for the destructor, copy constructor, and overloaded assignment operator, which must all be implemented if dynamically allocated memory is used in a class.

Generic Types

Many modern programming languages implement generic types in one way or another. Generic types allow functions and/or classes to be defined once for any type. This, along with function overloading and inheritance, is one of the ways modern languages implement polymorphism. In C++, generic functions and classes are defined using template parameter lists. For the basic syntax, we'll look at a short generic function that adds two values.

```
// generic functions and types have a template parameter
// list first that lists the alias for the generic type(s)
template <typename T>
// we then use T as our type alias
T add(const T &a, const T &b) {
    return a + b;
}
```

This function can be used to add together two members of the same type, provided they have `operator+` defined for them. At the end of this section we will briefly introduce concepts, which provide a mechanism for stating that the type `T` must have `operator +` defined.

Template Parameter Lists

When writing a generic function or data type, the signature must be preceded by the template parameter list. The template parameter list is started by the keyword `template`, followed by a list of template parameters in angle brackets. The parameter list can hold two constructs: template type aliases and non-type template parameters. First, let's discuss template type aliases. These are the most common use for templates, where you wish to define a single function or data type to operate on a variety of types. Template types in the parameter list have a "type" of either `typename` or `class`, although in modern C++ `typename` is preferred. The type alias can be any name (provided it's not a keyword or pre-existing name). When the type aliases have no semantic meaning, single, uppercase letters starting at `T` are used to name them. If the type alias has meaning, then you may name it something more expressive. Here is an example of a template parameter list with two template parameters: `template <typename T, typename U>`. Alternatively, `class` could be used in place of `typename` (again, this is not used as commonly now, especially post C++17): `template <class T, class U>`.

The next category of template parameters are non-type template parameters. Rather than aliasing a type, these parameters expect a value of a specific type. Additionally, the value passed for this parameter must be known at compile-time. An example of a template being used in this manner is the `std::array` data type. The `std::array` type expects two template parameters, one for the data type the array stores, and another for the size of the array. `std::array` is a wrapper around static, stack allocated arrays, so the size of the array must be known at compile time. This makes passing the array as a parameter to the constructor insufficient, as there is no way to force a parameter to the constructor to be a compile-time constant. When using a template parameter, the compiler will fail to compile the program if a compile-time constant is not provided for the size of the array. Here is an example of how the template parameter list for `std::array` would look: `template <typename T, size_t SIZE>`.

Declaring, Defining, and Using Generic

Functions

To declare a generic function, the template parameter list is provided before the function signature:

```
template <typename T>
void printValue(T val);
// putting the template parameter list on a separate line
// is more readable but not required
template <typename T> void printValues(T val1, T val2);
```

The template parameter list makes `T` available as a type name in the function that follows it. When defining the function, the template parameter list must be included again:

```
template <typename T>
void printValue(T val) {
    std::cout << val << std::endl;
}

template <typename T> void printValues(T val1, T val2) {
    std::cout << "Value 1: " << val1 << std::endl;
    std::cout << "Value 2: " << val2 << std::endl;
}
```

When calling the function, the type of the argument must be provided (C++17 adds the ability for the compiler to try to deduce the type of the argument, but it is still good practice to explicitly state the intended type):

```
int main() {
    // actual type of argument goes angle
    // brackets after the function's name
    printValue<int>(2);
    printValues<const char*>("hello", "world");
    return 0;
}
```

Declaring, Defining, and Using Generic Data Types

Declaring a generic class requires placing the template parameter list before the `class` keyword (structs can also be generic). The template

parameters declared before the class can be used anywhere within the class: as a data member's type, the return type of a function, the type of a parameter, etc. The one exception is for friends. In general, friend functions should be avoided as much as possible. Template friends, especially `operator<<` and `operator>>` are usually easier to defined inline within the class declaration, rather than outside the class, as is common in C++. However, in the next section we will see that this does not matter very much.

```

#include <iostream>
#include <stdexcept>

// declare a template parameter for the data type being stored
// and a non-type parameter for the size
template <typename T, size_t SIZE>
class StaticArray {
public:
    // constructor only has to set the size field
    // so we can define it in the class
    StaticArray() : size(SIZE) {}
    // returns the size of the array
    size_t getSize() const;
    // returns an array element (getter)
    // rather than having a separate function called at() like std::array
    // this will throw an out of range exception for invalid indices
    T operator[](size_t) const;
    // provides a reference to an array element (setter)
    // throws out of range exception for out of range access attempts
    T& operator[](size_t);
    // overloaded operator << to print the array
    // writing the definition of function outside of the class
    // declaration is very finicky
    friend std::ostream& operator<< (
        std::ostream& out,
        const StaticArray<T, SIZE>& arr
    ) {
        bool printComma = false;
        for (size_t i = 0; i < size; ++i) {
            if (printComma) std::cout << ", ";
            std::cout << data[i];
            printComma = true;
        }
    }
private:
    size_t size;
    // since size must be a compile-time constant we can do this
    T data[SIZE];
};

```

Like with the free functions, when defining the declared functions out of the class, we need to provide the template parameter list before each function. Additionally, when scoping the function to the class, the declared type alias must be passed to the class type. It will be easier to see what is meant by looking at the example.

```

// need to provide the template again
template <typename T, size_t SIZE>

```

```

// and pass to the class name before ::
size_t StaticArray<T, SIZE>::getSize() const {
    return size;
}

template <typename T, size_t SIZE>
T StaticArray<T, SIZE>::operator[](size_t index) const {
    if (index > size || index < 0) {
        throw std::out_of_range();
    }
    return data[index];
}

template <typename T, size_t SIZE>
T& StaticArray<T, SIZE>::operator[](size_t index) {
    if (index > size || index < 0) {
        throw std::out_of_range();
    }
    return data[index];
}

```

As can be seen in the example, each function needs to have the template parameter listed. This is needed to pass the declared template parameters to the template argument list of the class. In the next section, the compilation mechanisms of templates will be discussed.

Compiling Generics in C++

Earlier, we discussed that typically functions and classes are declared in a header file and implemented in a separate source file. This makes it possible to make changes to definitions without needed to recompile every file in the project. However, this is not possible with templates. Each time a generic type is created, or function is called, the compiler needs to generate a new version of the type/function's definition that replaces the template type alias with the actual type. Therefore, templates must be defined within a header file. It is possible to manually implement a template for a concrete type. This can be done in a source file.

Big 3

Unlike many other modern programming languages (C#, Java, Python), C++ allows for direct access to memory locations via pointers. When pointers are members of classes, it can create problems when allowing the compiler to automatically implement copy constructors,

destructors, and assignment operators for a class. First, let's look at the memory model of C++ programs (and really, most compiled programs).

C++ Program Memory Model

A computer program consists of five main areas in memory: text, data, heap, stack, and unmanaged. The text area contains the actual program instructions that need to be read and executed, along with constant values (constants are actually replaced by their actual, literal values by the compiler when the program is compiled). The data area contains static and global variables. The heap is used to store values that are dynamically allocated at runtime. The stack contains the values of local variables and the call stack. The unmanaged area is used for environment variables and command-line arguments.

Text Area

The text area contains the compiled instructions that describe what the program actual does. At this point, constants have been replaced by literal values and variables are replaced with their memory addresses. This is why declaring any value that could be constant as `const` is a performance optimization. The value becomes part of the program text, rather than needing to be read from memory anytime it is needed.

Data Area

The data area is separated into two regions for initialized and uninitialized values. The data area stores static and global variables. Statics and globals that are provided initial values are stored in the initialized data area. The uninitialized values are stored in what is commonly called the BSS (block start symbol) area. The initialized values area of the data area contains the variable offsets and values in the compiled executable file, whereas the BSS contains the amount of space required to hold the uninitialized variables. The variable are then allocated in the BSS when the program is executed.

Stack

The typical program diagram has the stack above the heap, but it makes more sense to explain the stack first to better differentiate the heap. This is not the same as the stack ADT we will discuss later in the course, but it does follow the same principle of last-in, first-out access. Each

time a function is called, its local variables are added to the top of the stack. When the function is done executing, all the variables allocated for that function are popped from the stack. The stack is managed by a combination of the compiler (when compiling the code) and the operating system (when the program is executing).

While the stack offers fast access and manages memory usage for the programmer, it is limited in size, and less flexible when allocating large amounts of memory due to access to the stack being in a specific order. Additionally, there may be instances where data needs to last longer than the lifetime of the function that creates it, but does not need to last the lifetime of the program (i.e., the variable is a poor candidate to be global or static).

Heap

To overcome the shortcomings of the stack, the heap is used. The heap is the largest area of memory allocated for a computer program (theoretically it could be the size of RAM minus what is needed for the stack, data, and text areas). While conceptually heap is often shown as growing from the bottom to the top of its "container", heap can actually allocate space anywhere, at any time. This makes it a much more flexible storage area than the stack. However, it also makes it slower to access, as the heap must be searched for an area large enough to hold the amount of data being requested. However, the heap makes runtime allocation of data possible, such as for raw pointers and dynamic arrays.

Destructor

Pointers can be assigned to the memory addresses of existing (stack allocated) variables, or used to create new memory on the heap to be used by the program. Memory on the heap is allocated using the `new` operator. Data created on the heap is freed after the pointer is done with it by using the `delete` operator. Failure to free memory results in memory leaks, where the operating system is unable to reclaim memory that is no longer in use by a program. The memory is

```
int main() {  
    // declare an integer pointer  
    int *ptr;  
    // ptr is a reference to a single integer on the heap
```



```

ptr = new int;
// use delete to free a single value
delete ptr;
// ptr is now a reference to a dynamic array on the heap
ptr = new int[10];
// use delete[] to free a
delete[] ptr;

int x;
// ptr is now pointing to a normally (stack) allocated variable
ptr = &x;
// pointers to existing variables are not deleted
return 0;
}

```

As you can see, `ptr` can either be a reference to a single integer in heap memory, an array of integers, or a stack allocated variable. This directly impacts which version of `delete` must be used to clean up the memory allocated for `ptr` in the heap. Additionally, `delete` cannot be called if the pointer is allocated on the stack. The compiler has no way to determine which version of `delete` to call in a destructor, making it impossible to effectively clean up an object that contains dynamic fields. Thus, implementing a destructor is required for any class that contains dynamic fields. A destructor has the same signature as a constructor, except it begins with a tilde `~`. Classes that do not use pointers to heap memory do not need to implement a destructor.

Copy Constructor

Copying also presents a challenge. Consider the following example:

```

int main() {
    int a = 4;
    int *x = &a;
    int *y = x;
    *y = 6;
    // now, a == 6
    // *x == 6
    // and *y == 6
    *x = 3;
    // now a == 3
    // *x == 3
    // and *y == 3
    return 0;
}

```

This example illustrates that pointers are references to a memory address. When we assign a pointer to another pointer, both pointers have ownership over that memory address. This creates a problem when copying class fields that are pointers. The compiler's default implementation of a copy constructor is to blindly copy all the fields in the class. This is fine if none of the fields are pointers, since they are all copied by value. However, when copying a pointer by value, the new object will be created with a copy of the *memory address* the pointer is referencing, not a copy of the data the pointer is referencing. Below is an example of a (not so) smart pointer wrapper class that does not implement a copy constructor, and an example of what will happen when copying an object of the class:

```
#include <assert.h>
// the SmartPointer class is meant to handle automatic
// allocation and deletion of a raw pointer so users do not
// have to remember to call new and delete
template <typename T>
class SmartPointer {
public:
    // default constructor allocates memory for the pointer
    SmartPointer() {
        ptr = new T;
    }
    // allocate memory and assign value
    SmartPointer(T val) {
        ptr = new T;
        *ptr = val;
    }
    // delete the pointer when the object goes out of scope
    ~SmartPointer() {
        delete ptr;
    }
    // allow accessing data with the * operator
    T operator*() const {
        return *ptr;
    }
    // allow assigning an int value to the pointer with =
    SmartPointer& operator=(T val) {
        *ptr = val;
    }
    // do not allow the object to be assigned to a raw pointer
    SmartPointer& operator=(T *other) = delete;
private:
    T *ptr;
};
```

```

int main() {
    // declare a smart pointer object
    SmartPointer<int> ptr;
    // assign its value to 34
    ptr = 34;
    // create a copy of ptr
    SmartPointer copy(ptr);
    // assign copy's value to 56
    copy = 56;
    // the value of ptr is now 56 since
    // copy is a duplicate of the actual pointer
    // not the value
    assert(*ptr == *copy);
    return 0;
}

```

The implementation for the copy constructor provided by the compiler looks something like this:

```

template <typename T>
SmartPointer<T>::SmartPointer(const SmartPointer& other) {
    ptr = other.ptr;
}

```

In reality, when we create a *new* smart pointer object via copying, we are probably expecting just the value to be copied, and for the two pointer objects to be independent entities. Even worse, when both objects go out of scope and the destructor is called, delete will be called on the same pointer twice. This will create a program crash due to a double free. Therefore, we must implement the copy constructor ourselves to avoid multiple objects from referencing the same memory location:

```

template <typename T>
SmartPointer<T>::SmartPointer(const SmartPointer& other) {
    ptr = new T;
    *ptr = *other;
}

```

In this implementation, the copy constructor allocates memory for the pointer using `new`, and then assigns the pointer the value being held by the other pointer, rather than copying the pointer itself.

Assignment Operator

Next, we need to consider the assignment operator, `operator =`. We cannot simply copy the fields of the object on the right-hand side of the equal sign over to the left-hand object. When the class is using dynamically allocated memory, we need to ensure that the left-hand object's memory is freed before copying over data from the right-hand object.

Copy and Swap Idiom

Defining an overloaded assignment operator the correct way allows the compiler to make several optimizations. The best way to overload `operator=` is to first create a separate member function that swaps two objects in constant time. Then, pass the right-hand object by value to `operator=`. This will invoke the copy constructor to create a copy of the right-hand object. Then, call the swap function inside `operator=`. This will swap the copy into the left-hand object, and whatever data was in the left-hand object into the parameter. Then, when the function ends, the destructor will be called on the parameter, which cleans up any old data in the left-hand object. Below is the `operator=` for the `SmartPointer` class.

```
// first define a constant time swap
template <typename T>
void SmartPointer<T>::swap(SmartPointer &other) {
    // vacuously a constant time swap of pointers
    // if these were dynamic arrays swapping the pointers
    // would save a lot of time
    T *temp = ptr;
    ptr = other.ptr;
    other.ptr = temp;
}
// you also need to return a reference to an object
// this way you can chain assignments: obj1 = obj2 = obj3;
template <typename T>
SmartPointer<T>& SmartPointer<T>::operator=(SmartPointer rhs) {
    swap(rhs);
    return *this;
}
```

Conclusion

Generic types in C++ allow one definition to be written for functions

and data types that use a multitude of input types. The Big 3 (destructor, copy constructor, and overloaded assignment operator) must be defined for any class using dynamically allocated memory.